COMP3821/9801

Workshop Week 7

Linear Programming and Approximation Algorithms

Copyright Disclaimer. *Reproducing, publishing, posting, distributing, translating this document is an infringement of copyright and will be referred to UNSW Student Conduct and Integrity for action.*

Guided Problem

The demonstrators will solve the following problems. For the rest of the workshop, work in small groups to solve the remaining workshop exercises. There are optional problems at the end of this document for further practice.

Guided Problem

Let $G = (V, E)$ be an undirected and weighted graph, where each vertex $v \in V$ has non-negative weight $w(v) \geq 0$. A *vertex cover* is a subset $S \subseteq V$ of vertices such that every edge is incident to at least one vertex in S. Given a graph $G = (V, E)$, your task is to find a vertex cover S whose total weight $\sum_{v \in S} w(v)$ is minimised. It is known that this problem is NP-hard.

(a) Write the integer linear program that captures this problem exactly; in particular, prove that each solution to the integer linear program describes a vertex cover and each vertex cover corresponds to a solution to the integer linear program.

(b) Describe and analyse a polynomial-time 2-approximation algorithm for the problem. Prove that your algorithm returns a feasible vertex cover and that it achieves an approximation ratio of 2.

Guided Problem

Let $G = (V, E)$ be an undirected graph. The *neighbourhood* of a vertex v is the vertex v and every vertex adjacent to v. A *double-dominating set* in G is a set $S \subseteq V$ of vertices such that, for each vertex $v \in V$, the neighbourhood of v contains at least two vertices in S.

Let $d > 2$, and let G be a graph such that every vertex has degree $d - 1$ (i.e. the neighbourhood of every vertex contains exactly d vertices). Furthermore, suppose that each vertex v has non-negative weight $w(v) \geq 0$. The goal is to find a double-dominating set S in G whose total weight $\sum_{v \in S} w(v)$ is minimised. It is known that this problem is NP-hard.

(a) Write the integer linear program that captures this problem exactly; in particular, prove that each solution to the integer linear program describes a double-dominating set and each double-dominating set corresponds to a solution to the integer linear program.

(b) Describe and analyse a polynomial-time $(d - 1)$-approximation algorithm for the problem. Prove that your algorithm returns a feasible double-dominating set and that it achieves an approximation ratio of $d - 1$.

Workshop Exercises

1. You are managing a company with a large number of project teams. The company consists of n employees. Each project team has exactly three employees and a single employee can belong to any number of project teams. Additionally, all of the project teams consist of different triplet of employees.

You need to organise a meeting to make a major announcement. Since this announcement affects every team, you need at least one member from each project team to be present at the meeting. Moreover, since the announcement is predicted to be unpopular among the employees, you want to avoid inviting employees who are likely to cause trouble.

For each employee x, let $\mathsf{trouble}(x) \in [0, 1]$ denote the probability that x is likely to cause trouble at the meeting. Your task is to choose a subset X of employees such that $\sum_{x \in X} \mathsf{trouble}(x)$ is as small as possible.

It is known that this problem is NP-hard.

(a) Write the integer linear program that captures this problem exactly; remember to prove the correspondence between solutions to the problem and integer solutions to the integer linear program.

(b) Describe and analyse a polynomial-time 3-approximation for the problem. Prove that your algorithm returns a feasible solution and that it achieves an approximation ratio of 3.

2. Suppose you are given an $n \times n$ binary array $A[1..n, 1..n]$; that is, the entries $A[i, j]$ of the array are either 0s or 1s. Each row of the array is coloured either red or blue, and you are given that there are exactly k 1s in each row and that each column j of the matrix has a non-negative weight $w(j) \geq 0$. Your goal is to choose a subset of columns that satisfy the following conditions.

   - For each red row i, *at least one* chosen column contains a 1 in row i.
   - For each blue row i, *at least two* chosen columns contain a 1 in row i.
   - The sum of the weights of the chosen columns is minimised.

   It is known that this problem is NP-hard.

   (a) Write the integer linear program that captures this problem exactly; in particular, prove that each solution to the integer linear program describes a feasible subset of columns and each subset of columns corresponds to a solution to the integer linear program.

   (b) Describe and analyse a polynomial-time k-approximation algorithm for the problem. Prove that your algorithm returns a feasible set of columns and that it achieves an approximation ratio of k.

3. We consider a variation to the weighted vertex cover problem, called the *prize-collecting weighted vertex cover* problem. In this problem, we are given an undirected and weighted graph $G = (V, E)$, where each vertex $v \in V$ has non-negative weight $w(v) \geq 0$ and each edge $e \in E$ has non-negative weight $q(e) \geq 0$. We want to return a subset $S \subseteq V$ whose total cost

$$\mathsf{cost}(S) = \sum_{v \in S} w(v) + \sum_{\substack{(u,v) \in E, \\ u,v \notin S}} q(u, v)$$

   is minimised. To understand this objective, we can think of the problem as trying to find a vertex cover but if S is *not* a vertex cover, then each edge that is not covered by S is penalised by its weight.

(a) Write the integer linear program that captures this problem exactly; remember to prove the correspondence between solutions to the problem and integer solutions to the integer linear program.

Hint. *You may need a variable for each vertex and a variable for each edge. It might help to think about the constraint as: if an edge is covered, then at least one of its endpoints must be chosen.*

(b) Describe and analyse a polynomial-time 3-approximation for the problem. Prove that your algorithm returns a feasible solution and that it achieves an approximation ratio of 3.

Hint. *Use the natural vertex cover rounding algorithm to obtain a 4-approximation algorithm first and then find ways to obtain a tighter analysis to obtain a 3-approximation. In particular, we do not need to use the threshold used by the vertex cover rounding algorithm.*